

PRODUCTION DEPLOYED

PlacementOS

Complete Frontend, Backend, Database and Integration Architecture

A detailed system-design blueprint of the deployed placement-preparation platform: presentation architecture, modular-monolith backend, relational data model, critical workflows, AI/media orchestration, real-time notifications, security boundaries and deployment topology.

Frontend: React 19 + TypeScript + Vite

Backend: Node.js + Express + TypeScript

Database: Neon PostgreSQL + Prisma 7

Realtime: Socket.IO

AI: Groq

Media: Browser FFmpeg + Cloudinary

Architecture style

Domain-oriented modular monolith. One backend deployment with explicit route, controller, service, persistence and adapter boundaries.

Product operating model

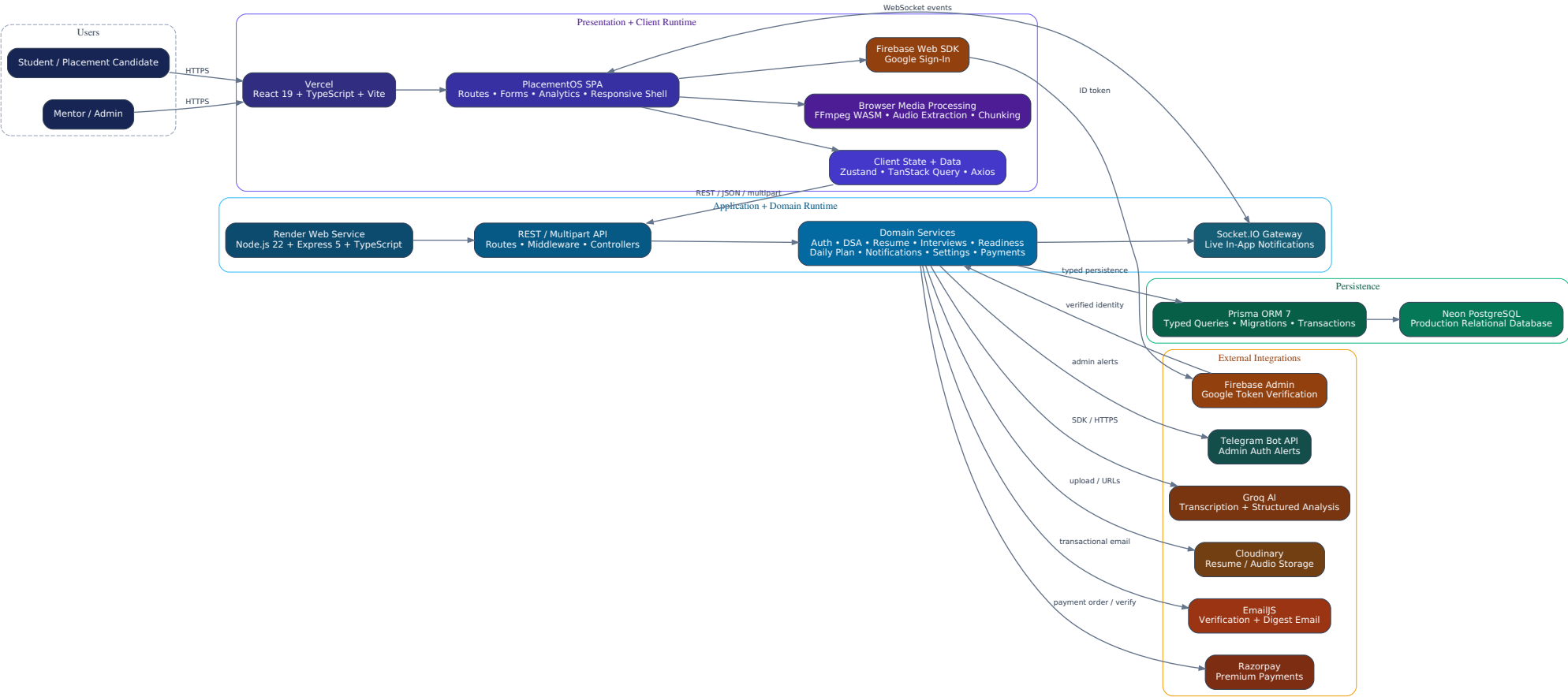
Evidence → diagnosis → readiness signal → prioritized action → review. DSA, resume and interview data are connected instead of isolated.

Current deployment

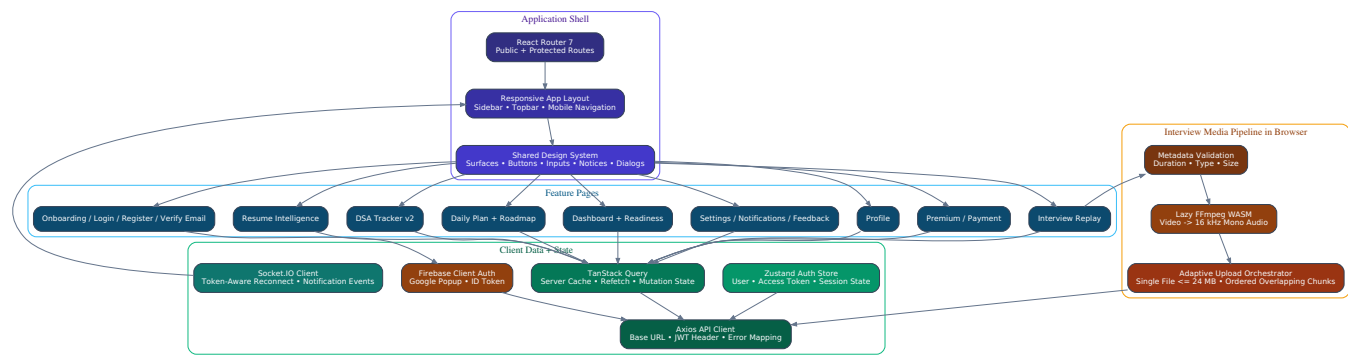
Vercel SPA → Render API → Neon PostgreSQL, with Firebase, Groq, Cloudinary, EmailJS, Razorpay and Telegram integrations.

Scope note: This blueprint focuses on the pure deployed project architecture. External scheduling infrastructure is intentionally excluded from the current diagram set.

1. Complete System Context and Deployment Topology



2. Frontend Architecture - React SPA



Presentation layer

React Router separates public and protected routes. A shared responsive shell provides desktop sidebar, topbar and mobile navigation. Reusable design primitives enforce the onboarding-derived navy/violet visual system.

Data and state

TanStack Query manages server cache and mutation lifecycle; Zustand stores authentication/session state; Axios centralizes API base URL, bearer-token injection and error handling.

Realtime

The Socket.IO client reads the latest access token on every reconnect, joins the authenticated user room and receives notification-completion events without page polling.

Browser media processing

Video never needs to reach the server. FFmpeg WebAssembly extracts compressed mono audio locally, and the uploader chooses single-file or ordered overlapping chunks based on processed size.

3. Frontend Module Responsibilities

Frontend area	Primary responsibility	Key technology / behavior	Backend dependency
Onboarding and auth	Registration, login, email verification, Google sign-in, route protection	Firebase Web SDK, auth store, form validation, JWT persistence	Auth API, Firebase Admin token verification, verification email
Dashboard	Readiness overview, domain scores, streak, starter plan, company readiness	TanStack Query cache, responsive cards, status-aware empty states	Readiness, profile, DSA, resume, interview, streak endpoints
DSA Tracker v2	Problem tracking, topic/pattern coverage, revisions, notes and company tags	Typed forms, filters, analytics and revision queue	DSA problem/revision services and readiness recalculation
Resume Intelligence	PDF upload, ATS analysis, role-fit feedback, score display	Multipart upload, analysis loading states, actionable feedback UI	Cloudinary, Groq analysis, Resume persistence, readiness update
Interview Replay	Manual/audio/video interview capture and structured coaching	FFmpeg extraction, chunk planning, upload orchestration, replay rendering	Transcription, AI analysis, QuestionReplay persistence, Socket.IO completion
Daily Plan and Roadmap	Prioritized preparation actions and regeneration	Task cards, completion state, bounded AI-generated plans	Readiness signals, profile targets, Groq planning service
Settings	Notification preferences, account controls, support and feedback	Toggle groups, timezone/time selection, confirmation notices	NotificationPreference, Feedback and account APIs
Premium	Plan display and payment flow	Razorpay checkout integration and transaction state	Payment order creation and signature verification

Accessibility

Semantic controls, visible focus, reduced-motion handling, mobile touch targets and explicit loading/error states.

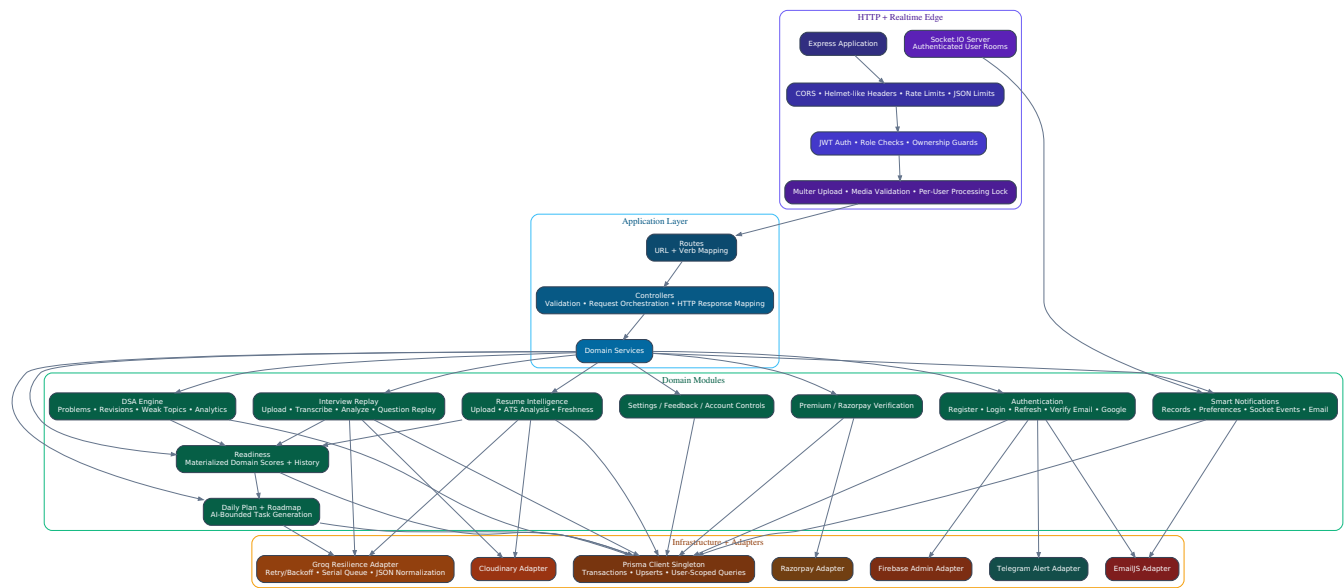
Build and routing

Vite production build on Vercel with SPA rewrite to index.html for direct protected-route refreshes.

Environment contract

Only browser-safe VITE_* configuration is exposed: API/socket URL, Firebase web configuration and support email.

4. Backend Architecture - Domain-Oriented Modular Monolith



Edge and middleware

Express handles HTTP requests while middleware enforces CORS, authentication, authorization, rate limits, upload validation and duplicate-processing protection.

Controllers

Controllers translate transport concerns into domain commands: parse request data, call services, map provider failures to safe HTTP statuses and return stable response shapes.

Domain services

Business rules live in services rather than routes. Cross-domain services update readiness, create notifications and preserve user ownership.

Infrastructure adapters

Prisma, Groq, Cloudinary, Firebase, EmailJS, Razorpay and Telegram are isolated behind adapter/service boundaries to reduce provider coupling.

5. Backend Domain Responsibilities and Reliability Controls

Domain modules

Module	Core invariants
Authentication	Unique identity, hashed password, verified Google token, short-lived access token, refresh-token flow, user-scoped profile creation.
DSA	Every problem belongs to one user; revisions belong to one problem; status/topic/pattern updates feed analytics and readiness.
Resume	Only validated PDFs are accepted; external object metadata and analysis are persisted; latest evidence affects readiness.
Interview	Source type and duration limits are validated; chunk order is deterministic; one coherent transcript drives structured analysis.
Readiness	Domain contributions are normalized and materialized; history snapshots preserve score evolution.
Notifications	Preferences determine eligibility; dedupe keys avoid repeated in-app records; Socket.IO publishes live completion events.

Reliability and error boundaries

AI retry wrapper

Retries transient 429/5xx/network failures with exponential backoff, jitter and Retry-After support.

Serial transcription queue

Protects shared Groq free-tier capacity and preserves ordered processing for multi-chunk media.

Per-user processing lock

Rejects duplicate interview processing before upload middleware consumes memory or spends AI capacity.

Safe provider mapping

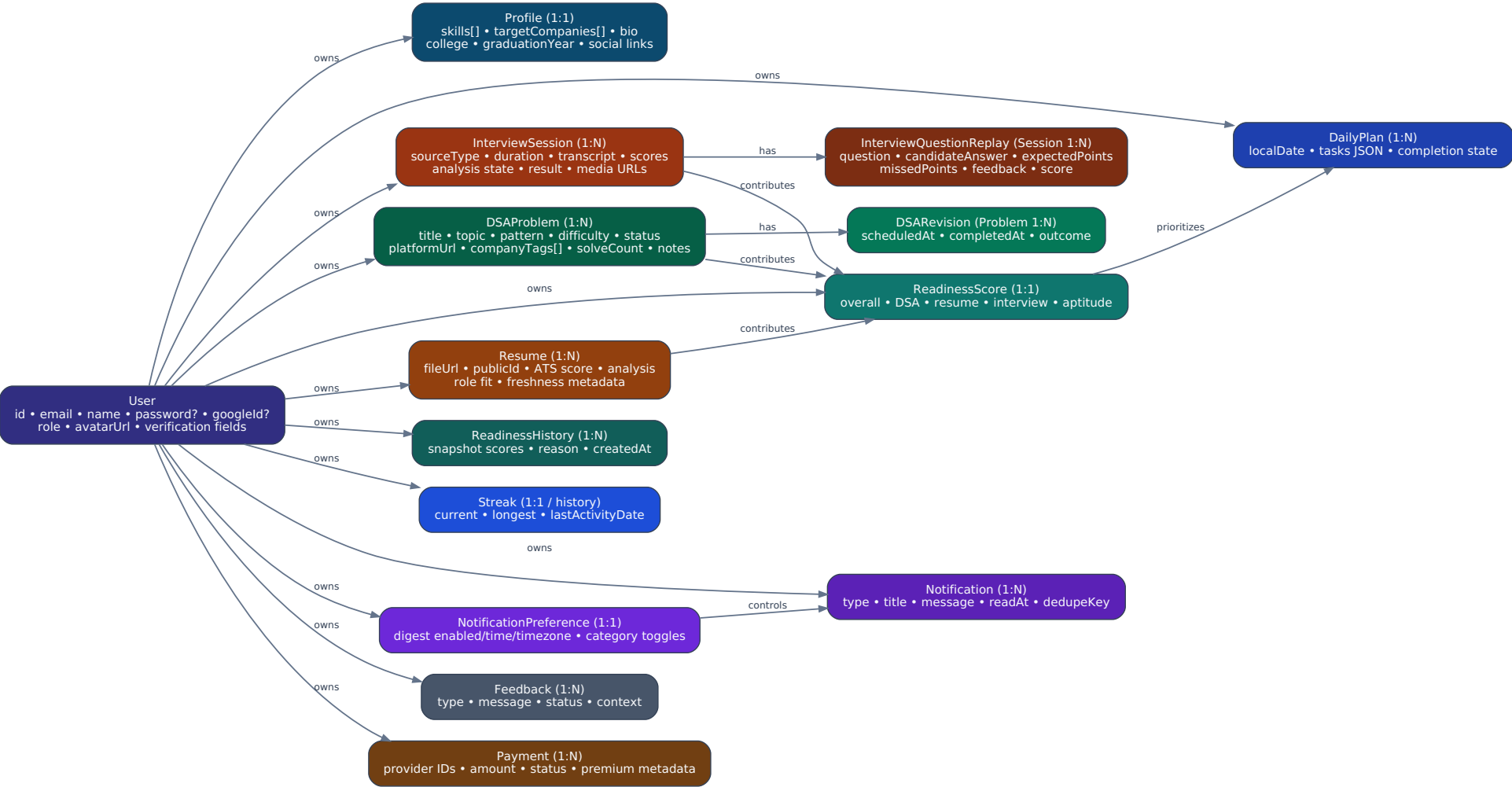
Rate limit → 429, provider/network outage → 503, duplicate work → 409, validation → 400, unauthorized → 401/403.

Build and deployment

TypeScript compilation, Prisma generation/migrations and npm start are executed through Render deployment.

Scaling boundary: the current lock and AI queue are process-local and correct for one Render instance. Multi-instance scaling would require Redis-backed coordination or a durable worker queue.

6. Database Architecture - Neon PostgreSQL + Prisma



7. Relational Design and Data Ownership

Core data-design principles

- **User as aggregate root:** all preparation evidence is user-owned, enabling authorization through user-scoped Prisma queries.
- **One-to-one state:** Profile, ReadinessScore and NotificationPreference are stable aggregates usually maintained through upsert operations.
- **One-to-many evidence:** DSA problems, resumes, interviews, plans, notifications, feedback and payments preserve history instead of overwriting the user record.
- **Structured AI output:** InterviewQuestionReplay records make questions, candidate answers, missed points and feedback queryable rather than storing one opaque response.
- **Materialized readiness:** dashboard reads remain predictable because current scores are stored, while ReadinessHistory records explain change over time.
- **Migration discipline:** Prisma migrations version schema evolution and are deployed against Neon using the direct connection.

Production tables

Category	Tables
Identity	User, Profile
DSA	DSAProblem, DSARevision
Resume	Resume
Interview	InterviewSession, InterviewQuestionReplay
Readiness and planning	ReadinessScore, ReadinessHistory, DailyPlan, Streak
Engagement	Notification, NotificationPreference
Operations	Feedback, Payment, _prisma_migrations

Consistency strategy

Cross-domain write operations use service-level orchestration: persist the source evidence, recalculate the affected readiness contribution, write history where required, and publish a notification only after successful persistence.

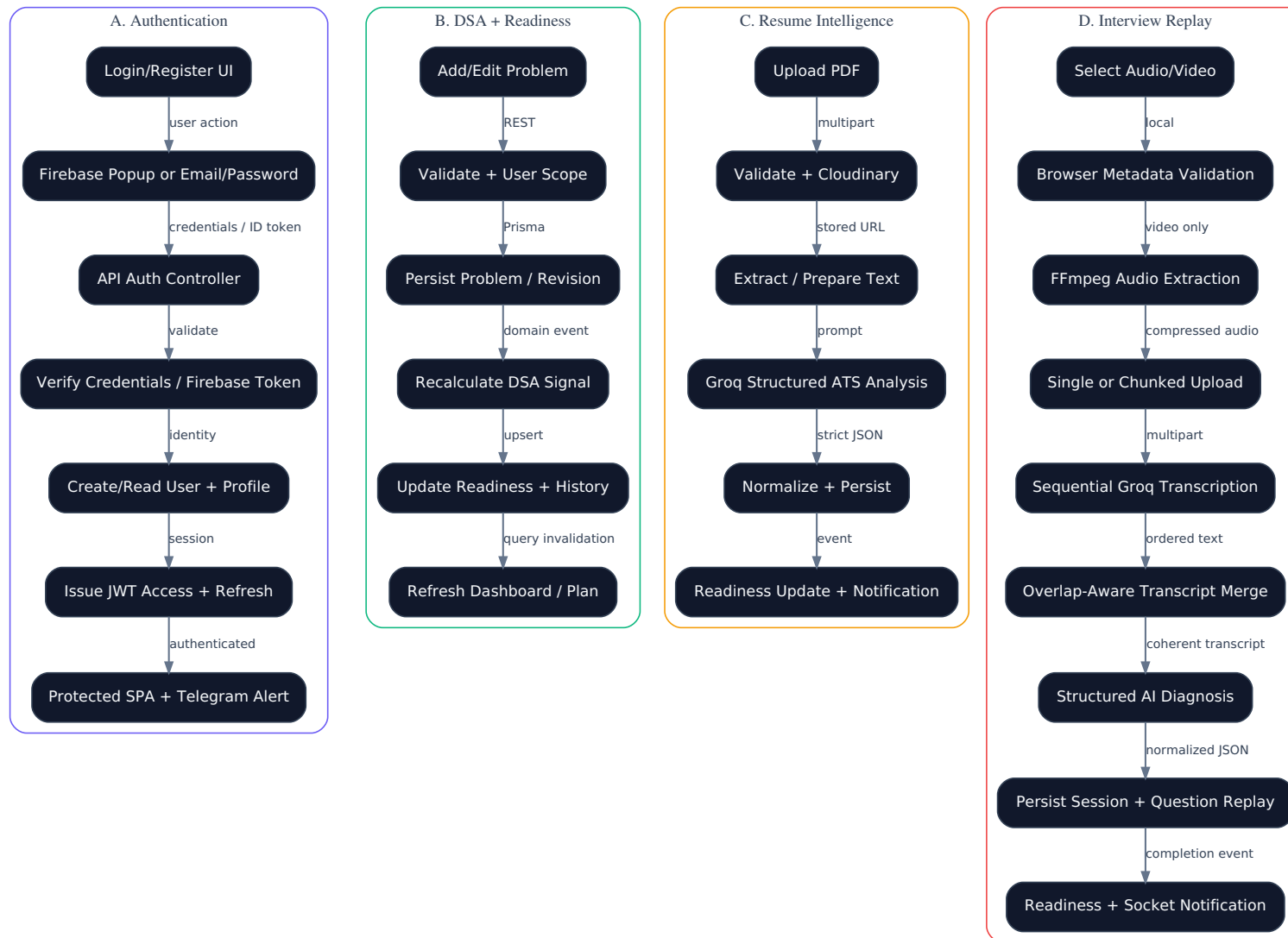
Security strategy

Record ownership is verified in every user-facing query. Database credentials remain server-only. Vercel receives no database secret; Render uses pooled runtime connectivity and a direct URL only during migrations.

```
Browser / Vercel
-> Render API
-> Prisma Client
-> Neon pooled runtime connection

Render build migration
-> Prisma migrate deploy
-> Neon direct connection
```


8. Critical End-to-End Workflows



9. Security Boundaries, Deployment and Operational Model

Browser boundary

Only VITE_* public configuration is shipped. JWT access tokens are attached by the API client. Original interview video stays local; only processed audio is transmitted.

API boundary

JWT middleware, role checks, CORS origin allow-list, rate limits, validation and ownership-scoped queries protect application commands.

Provider boundary

Groq, Cloudinary, Firebase Admin, EmailJS, Razorpay and Telegram credentials are stored only in Render environment variables.

Frontend deployment

Vercel builds the client directory with npm ci and npm run build, serves dist, and rewrites all SPA routes to index.html.

Backend deployment

Render installs dev dependencies, generates Prisma Client, applies migrations through the direct Neon URL, compiles TypeScript and runs node dist/index.js.

Database deployment

Neon hosts the production PostgreSQL database. Runtime traffic uses a pooled URL; schema migration uses a direct URL.

Current production paths

Component	Deployment	Responsibility
Web client	Vercel - placement-os-kappa.vercel.app	Static SPA assets, browser runtime, direct-route rewrite
API server	Render - placements-api-2acg.onrender.com	REST, multipart, Socket.IO, AI orchestration and integrations
Database	Neon PostgreSQL - Singapore region	Persistent relational state and migrations

Operational scope: external scheduler configuration is not part of this blueprint. The deployed product remains fully usable for interactive frontend/backend/database workflows and real-time completion events.

10. How to Explain the Architecture in an Interview

30-second explanation

PlacementOS is a React and TypeScript SPA deployed on Vercel, backed by a Node/Express modular monolith on Render and a Neon PostgreSQL database accessed through Prisma. DSA, resume and interview evidence share one user-centric relational model and feed a materialized readiness score. REST handles commands and data, while Socket.IO pushes analysis-completion notifications. The most advanced workflow processes interview video in the browser with FFmpeg, uploads only compressed audio, transcribes it through Groq with retry and queue controls, persists structured question-level feedback, and updates readiness.

Why a modular monolith?

The domains are strongly connected and share transactions, but the product is operated by a small team. A modular monolith keeps deployment and consistency simple while preserving boundaries that can later become workers or services.

Strongest architecture decision

Client-side video-to-audio extraction reduces privacy exposure, backend bandwidth and storage cost while still enabling transcription and coaching.

Most important data decision

AI output is normalized into relational records instead of stored as one response blob. This makes the product queryable, supports analytics and allows readiness recalculation.

Reliability decision

External AI calls are wrapped with retry/backoff and safe status mapping; transcription is serialized and duplicate interview work is rejected per user.

Known limitation and evolution

The lock and queue are process-local. At higher scale, the next step is a durable AI/media job worker with Redis coordination, idempotency keys, structured observability and asynchronous 202 Accepted workflows.

Architecture quality summary

Quality	Current strength	Next evolution
Maintainability	Typed full stack, domain services, adapter boundaries	Central policy modules and contract tests
Reliability	Retry, queue, locks, validation and safe errors	Durable jobs, distributed locks and dead-letter handling
Privacy	Original video excluded by architecture	Object lifecycle reconciliation and retention controls
Scalability	Stateless REST + relational persistence	Worker queue, Redis semaphore and streaming uploads
Operability	Vercel/Render/Neon deployment, health endpoints, build gates	Structured logs, metrics, traces and admin monitoring